F1 GP WINNER PREDICTION SYSTEM

Technical Overview & Project Documentation

FastF1 | FastAPI | React | scikit-learn

Nishanth Beeram
beeram.nishanthreddy@gmail.com
June 2026

1. Project Overview

The F1 GP Winner Prediction System is a full-stack machine learning application that predicts the probability of each driver winning a Formula 1 Grand Prix. It combines historical race data, qualifying times, season statistics, live practice session telemetry, and a circuit-specific chaos model to produce ranked win probabilities with best-case / likely / worst-case scenario ranges.

Live URLs:

- **Frontend:** https://f1-gp-predictor.netlify.app
- **Backend API:** https://f1-gp-predictor.onrender.com

Project Phases:

- **Phase 1:** Core ML pipeline - data ingestion from FastF1, feature engineering, model training on 2018-2024 seasons.
- **Phase 2:** Stripped to winner-only prediction; removed lap time model; LogisticRegression AUC = 0.9534.
- **Phase 3:** React frontend with dark telemetry-HUD aesthetic; animated probability bars; podium cards.
- **Phase 4:** Practice telemetry pipeline (FP1/FP2/FP3); chaos matrix; best/likely/worst scenario ranges.

2. Technology Stack

Layer	Technology	Version	Purpose
Data	FastF1	3.8.3	F1 telemetry, timing, weather from official F1 API
Data	pandas / numpy	latest	Data manipulation and numerical operations
ML	scikit-learn	latest	Logistic Regression + Gradient Boosting models
ML	joblib	latest	Model serialisation
Backend	FastAPI + uvicorn	latest	REST API - async, auto-docs at /docs
Backend	Python	3.11	Primary language for all backend and ML code
Frontend	React 18 + Vite	18	Component-based UI with fast build tooling
Frontend	Tailwind CSS	latest	Utility-first CSS - dark telemetry HUD theme
Frontend	lucide-react	latest	Icon library (Database, Zap, Trophy icons)
Hosting	Render	Free tier	Backend deployment - 512MB RAM, sleeps after 15 min
Hosting	Netlify	Free tier	Frontend - auto-deploys on every git push to main
Version Control	Git / GitHub	-	github.com/Nishanth784/f1-gp-predictor

3. System Architecture

Component	Description
FastF1 (Data Source)	Connects to F1's official timing data and telemetry streams. Returns session objects with laps, telemetry channels, weather, and track status events.
data_ingestion.py	Wrapper around FastF1. Fetches schedules, qualifying, race results. Enables local disk cache (fastf1_cache/) to avoid re-downloading.
Feature Engineering	Transforms raw session data into ML-ready numerical features. Merges practice telemetry cache and adds chaos scalars per race.
winner_model.py	Trains LogisticRegression + GradientBoosting. Saves best model to models/winner_model.joblib.

Component	Description
FastAPI Backend	Loads saved model at startup. Serves predictions via REST. Runs practice precompute as a background task.
React Frontend	Calls the FastAPI backend. Renders ranked driver cards with scenario bars, chaos tiles, and practice data badge.
Practice Cache (cache/*.json)	JSON files written by batch job, read by backend at request time. Optional - predictions work without them but are richer when present.

4. File-by-File Breakdown

Every significant file in **D:\prediction system** and its role:

File	Role
main.py	Training entry point. Loops 2018-2024, collects features + labels for every GP, calls train_and_evaluate_winner_model(), saves model. Tees stdout to training_log.txt.
data_ingestion.py	FastF1 wrapper: get_event_schedule(), get_race_results(), get_qualifying_results(). Enables disk cache, suppresses noisy warnings.
winner_feature_engineering.py	Core feature pipeline. Takes raw qualifying + race data, engineers all 75+ features, merges practice cache if available, adds chaos scalars per race.
winner_model.py	Model training and prediction. Trains GBC and LR, picks best by AUC, saves to models/winner_model.joblib. align_winner_features_to_model() fills missing columns with 0.0 for backward compatibility with new features.
winner_labels.py	Generates IsWinner binary labels: 1 = race winner, 0 = everyone else.
race_aggregation.py	calculate_degradation_slope() fits linear regression on tyre stint lap times to quantify degradation rate. Also provides race-level aggregation utilities.
practice_data_ingestion.py	NEW. Batch job to extract FP1/FP2/FP3 telemetry. Loads sessions via FastF1 (laps + telemetry + weather). Computes brake point deltas field-relative. Saves to cache/YYYY_gp_name_practice.json. CLI: python practice_data_ingestion.py 2026 'Austrian Grand Prix'
practice_feature_engineering.py	NEW. Long run detection: 4+ consecutive clean laps on same tyre set. Computes race pace per compound (AvgPace, BestPace, DegradationSlope, Consistency). Adds _Gap columns showing each driver vs field average.
chaos_matrix.py	NEW. Static SC rate lookup per circuit (Monaco 85%, Monza 35%). Reads FastF1 track_status codes 4=SC / 6=VSC for real historical SC detection. compute_chaos_index() blends SC rate + grid spread + wind. compute_scenario_ranges() generates best/likely/worst probability bands.
precompute_all_practice.py	NEW. Bulk batch job. Loops all GPs in a year range, extracts and caches practice data. Skips already-cached GPs. Falls back to hardcoded calendar if FastF1 schedule API is down.
backend/main.py	FastAPI app. Loads model at startup. All prediction endpoints. Returns DriverPrediction objects with scenarios (best/likely/worst), chaos_index, sc_rate, has_practice_data flag.
frontend/src/pages/Results.jsx	Main results page. ScenarioBar: 3-layer bar (faint best extent, shaded range band, solid likely fill). Stats strip: Chaos Index, SC Probability, Drivers, Top Pick. PodiumCard shows WORST%/BEST% labels. PRACTICE DATA badge shown when cache present.
frontend/src/pages/Home.jsx	Landing page with year + GP selector. Calls /years and /schedule.
train_model.bat	Double-click to retrain locally via venv Python. 20-40 min. Progress logged to training_log.txt.
precompute_all_practice.bat	Bulk-caches practice data for 2024+2025+2026. Skips cached GPs.
git_push.bat	Stages all changes, commits, pushes to GitHub. Render and Netlify auto-deploy.
models/winner_model.joblib	Serialised trained model. Committed to git so Render loads it on deploy.
cache/*.json	Practice feature caches (one per GP). NOT in git - local only. Auto-used by backend when present.

5. FastF1 - What It Is and What We Use

FastF1 is an open-source Python library wrapping the official F1 timing data API, historical databases, and telemetry streams. It is the standard tool used by F1 engineers and analysts to access race data.

What FastF1 provides and what we take from it:

Data Type	FastF1 Fields Available	Used in This Project
Lap Times	LapTime, Sector1/2/3Time, IsPersonalBest	BestLapTime, sector bests, LapCount
Tyre Data	Compound (SOFT/MEDIUM/HARD), TyreLife, FreshTyre	Race pace per compound, degradation slope
Telemetry (10Hz)	Speed, Throttle, Brake, Gear, DRS, Distance, RPM	All except RPM (excluded - see below)
Weather	AirTemp, TrackTemp, WindSpeed, WindDirection, Humidity	All - averaged across FP sessions
Track Status	Codes: 1=Green, 4=SC, 6=VSC, 5=Red Flag	Safety car probability per circuit
Qualifying	Q1/Q2/Q3 times, grid position, driver + team	Core prediction features
Race Results	Final position, points, DNF, driver + team	Labels + season stats
Event Schedule	GP name, date, circuit, round number	Loop training data collection

What FastF1 does NOT provide:

- Fuel loads - not telemetered publicly. Engine management decides fuel mapping privately.
- Engine modes / power modes - team-internal settings, never published.
- Car setup data - ride height, wing angles, camber, suspension - not available via any public API.
- Pit stop strategy (planned) - only actual pit stop lap numbers recorded, not the plan.

Why RPM was excluded from telemetry features:

RPM varies heavily with fuel load. A car at the start of a practice session (heavy fuel) shows much lower RPM at the same throttle position than at the end. Since we only extract from the fastest lap (which can occur at any fuel state), RPM adds noise rather than signal. Speed, throttle percentage, and DRS deployment are far more stable proxies for aerodynamic setup and driver confidence.

6. All Features - Complete List

The model uses features across 5 groups. Base set (75 features) always available. Chaos features always added. Practice features added when cache exists for the GP.

Group 1 - Qualifying and Grid

Feature	Description
GridPosition	Starting grid position (1 = pole)
GridPositionNorm	Grid position normalised 0-1 across field
Q1Seconds	Q1 lap time in seconds (0 if not set)
Q2Seconds	Q2 lap time in seconds (0 if eliminated in Q1)
Q3Seconds	Q3 lap time in seconds (0 if eliminated in Q1/Q2)
BestQualiTime	Best time across Q1/Q2/Q3 in seconds
QualiTimeGap	Gap in seconds to pole position time

Group 2 - Season Statistics (rolling, computed up to race day)

Feature	Description
DriverWinRate	Driver's win rate in races so far this season
DriverAvgPosition	Driver's average finishing position this season
DriverTotalWins	Driver's total wins so far this season
TeamWinRate	Constructor's win rate so far this season
TeamAvgPosition	Constructor's average finishing position this season
TeamTotalWins	Constructor's total wins so far this season

Group 3 - Identity Encoding (approx. 65 features)

Every unique driver and constructor in 2018-2024 training data is one-hot encoded. This gives the model implicit knowledge of each driver's and team's historical performance beyond rolling stats - the 'VER' column implicitly carries Verstappen's dominance history.

Group 4 - Chaos Features (same value for all drivers in a race)

Feature	Description
CircuitSCRate	Probability of a Safety Car at this circuit. Computed from FastF1 track_status codes (4=SC, 6=VSC) across historical races. Static fallback: Monaco 85%, Azerbaijan 80%, Singapore 75%, Monza 35%, Spain 30%.
GridSpread	P1-to-P10 qualifying gap in seconds, normalised 0-1. Tight grid = high chaos. Less than 1.0s gap = 1.0 (maximum chaos). More than 2.5s gap = 0.0.
AvgWindSpeed_chaos	Session average wind speed from weather data.
ChaosIndex	Composite score: (CircuitSCRate + GridSpread + WindFactor) / 3. Drives the scenario range width - high chaos means wider best/worst spread.

Group 5 - Practice Telemetry Features (from cache when available)

Only present when practice_data_ingestion.py has been run for the GP. Defaults to 0.0 if missing. Model still runs correctly without them.

Feature	Source	Description
MaxTrapSpeed	Telemetry	Highest speed on driver's fastest lap (km/h). Proxy for aero downforce level.
FullThrottlePct	Telemetry	% of fastest lap at 100% throttle. Higher = more power-dependent circuit.
BrakingPct	Telemetry	% of fastest lap with brakes applied. Higher = more stop-start circuit.
DRSDeploymentPct	Telemetry	% of fastest lap with DRS open (codes >=10). Proxy for straight-line opportunity.
BrakePointDelta	Telemetry	Driver's first brake distance vs field average. Positive = brakes later = more grip confidence.
AvgAirTemp	Weather	Average air temperature across FP1/FP2/FP3 (degrees C).
AvgTrackTemp	Weather	Average track surface temperature across FP sessions (degrees C).
AvgWindSpeed	Weather	Average wind speed across FP sessions (km/h).
TrackTempDeltaFP1toFP3	Weather	Track temp change FP1 to FP3. Positive = track rubbers in and gets grippier.
SOFT_AvgPace	Race pace	Average lap time on SOFT compound during long runs (seconds). FP2 primary.
SOFT_BestPace	Race pace	Fastest long-run lap on SOFT.
SOFT_DegradationSlope	Race pace	Rate of lap time increase per lap on SOFT (s/lap). Higher = faster degradation.
SOFT_Consistency	Race pace	Lap time std deviation on SOFT. Lower = more consistent.
MEDIUM_AvgPace	Race pace	Average long-run pace on MEDIUM.
MEDIUM_BestPace	Race pace	Best long-run lap on MEDIUM.
MEDIUM_DegradationSlope	Race pace	Degradation rate on MEDIUM.
MEDIUM_Consistency	Race pace	Consistency on MEDIUM.
HARD_AvgPace	Race pace	Average long-run pace on HARD.
HARD_BestPace	Race pace	Best long-run lap on HARD.
HARD_DegradationSlope	Race pace	Degradation rate on HARD.
HARD_Consistency	Race pace	Consistency on HARD.

Long run detection: a stint qualifies if it has 4+ consecutive clean (non-in/out) laps on the same tyre set, using TyreLife field to detect stint boundaries. Filters out flying laps.

7. Machine Learning Model

The task is **binary classification**: for each driver, predict P(IsWinner=1) - the probability this driver wins the race.

Model Selection

Model	AUC-ROC	Notes
Logistic Regression (CURRENT BEST)	0.9534	Best generalisation. Simpler, less prone to overfitting on small per-race sample sizes. This is the production model.
Gradient Boosting Classifier	0.9560	Slightly higher training AUC but overfits more. Kept as backup.

AUC-ROC 0.9534 means: given any random pair of (winner, non-winner), the model ranks the winner higher 95.3% of the time. Strong result for a sport as unpredictable as F1.

Training Details

- Training data: 2018-2024 seasons (2025 failed to load from FastF1 API during training).
- Samples: 2,979 driver-race entries. Each row = one driver in one GP with all features.
- Labels: IsWinner=1 for race winner, 0 for all others. Roughly 1:20 class imbalance.
- Feature alignment: align_winner_features_to_model() fills any new features (practice/chaos) not seen during training with 0.0 so the model still runs without error.

Inference Pipeline

1. Load qualifying results and current season stats for the selected GP.
2. Engineer features (Groups 1-4). Load practice cache if it exists (Group 5).
3. Align feature columns to model schema - fill missing with 0.0.
4. model.predict_proba(X) returns raw win probabilities per driver.
5. Apply chaos adjustment: smooth probabilities toward uniform distribution proportional to ChaosIndex.
6. Compute scenario ranges. Return ranked results.

Scenario Ranges

Scenario	Formula	What It Means
best_case	95% x raw + 5% x (1/N drivers)	Everything goes right for this driver. Minor upward lift for mid-field.
likely	Chaos-adjusted probability	The headline prediction. Smoothed for circuit unpredictability.
worst_case	Blend toward uniform using chaos x 1.5 (capped at 0.9)	Maximum chaos assumed. Favourite's odds compressed. Field equalised.

8. API Endpoints

Base URL: **https://f1-gp-predictor.onrender.com**

Endpoint	Method	Description
GET /health	GET	Health check. Returns {status: ok}. Pinged to prevent Render sleep.
GET /years	GET	Returns [2018, 2019, ..., 2026]. Populates year dropdown.
GET /schedule?year=	GET	Returns list of GP names for the year. Populates GP dropdown.
GET /winner-probabilities?year=&gp;=	GET	Full ranked prediction. Returns drivers sorted by likely probability with best/likely/worst scenarios, chaos_index, sc_rate, has_practice_data.

Endpoint	Method	Description
POST /predict-winner	POST	Body: {year, gp, driver (optional)}. Same as GET endpoint.
POST /precompute-practice/{year}/{gp}	POST	Kicks off FP1/FP2/FP3 extraction as background task. Returns immediately. Saves cache after 10-45 min.
GET /practice-status/{year}/{gp}	GET	Checks if practice cache exists. Returns {available, drivers, feature_count}.

9. Frontend

React 18 SPA, built with Vite, styled with Tailwind CSS, deployed on Netlify. Two pages:

Home Page (/)

- Year dropdown from GET /years (2018-2026). GP dropdown from GET /schedule - updates when year changes.
- Displays: training range (2018-2025, 7 seasons), 20+ features per driver, live status.

Results Page (/results/:year/:gp)

- **ScenarioBar:** Three-layer bar. Faint background = best-case extent. Semi-transparent band = worst-to-best range. Solid fill = likely probability.
- **Stats strip (4 tiles):** CHAOS INDEX (0-1), SC PROBABILITY (%), DRIVERS (count), TOP PICK (name).
- **Podium Cards (Top 3):** Large cards with driver name, team, likely%, and WORST X% / BEST X% labels.
- **Driver table:** All other drivers with ScenarioBar and range label.
- **PRACTICE DATA badge:** Appears when has_practice_data=true. Database icon. Indicates telemetry features used.
- **Footnote:** Plain-English explanation of best/likely/worst scenarios.

10. Deployment

Item	Detail
Backend host	Render free tier - 512 MB RAM, 0.1 CPU. Sleeps after 15 min of inactivity.
Cold start	~50 seconds on first request after sleep. UptimeRobot pings /health every 14 min to keep awake.
Python version	Set PYTHON_VERSION=3.11.0 manually in Render dashboard Environment Variables.
Model on Render	models/winner_model.joblib committed to git. Render loads it at startup. Retraining requires local run then git push.
Frontend host	Netlify free tier. Auto-deploys on every push to main. Build: npm run build. Publish: dist/
Practice caches	NOT on Render (RAM limited). Local only. Run precompute_all_practice.bat locally.
Redeployment	Run git_push.bat -> GitHub -> Render rebuilds backend + Netlify rebuilds frontend.

11. Live Race Weekend Workflow

When	Action	How
After FP3 Saturday	Run practice precompute	Double-click precompute_all_practice.bat (or run .py with 2026)
Wait 15-45 min	Telemetry extracts in background	Watch the terminal for [GP OK] messages
After Qualifying	Open the site	f1-gp-predictor.netlify.app -> 2026 -> GP name

When	Action	How
Check badge	PRACTICE DATA badge appears if cache ready	Top-right of results page
Race day	Predictions remain valid	Refresh to recheck at any time

Without precompute: predictions still work using qualifying times and season stats only. Practice data is optional but gives richer predictions.

12. Future Scope

Near-Term Improvements

- **Weather forecast integration:** Pull race-day rain probability (e.g. OpenMeteo API). Wet races have wildly different outcomes - a rain probability flag could heavily adjust the chaos index.
- **Pit stop strategy modeling:** FastF1 records which lap each driver pitted. Build a feature predicting undercut vs overcut likelihood based on pace trajectory.
- **Driver form / momentum:** Rolling 5-race form index instead of season averages. A driver on a hot streak signals differently than their season average suggests.
- **Grid penalty detection:** Grid penalties shift positions artificially. Treating penalised grid positions differently from true qualifying pace would improve accuracy.

Medium-Term

- **Automatic practice precompute:** Schedule a cron job on a paid tier or VPS to run precompute automatically after each FP3 without manual intervention.
- **Multi-race prediction:** Season-level model predicting championship outcomes probabilistically.
- **Cloud practice cache storage:** S3 or similar so Render can serve practice-enriched predictions without relying on local machine caches.

Ambitious / Long-Term

- **Live timing integration:** Update predictions lap-by-lap during the race as positions change.
- **Driver comparison tool:** Head-to-head analysis - 'who wins if it rains, if there is a safety car, if Verstappen starts P5?'
- **Bayesian model:** Replace scikit-learn with PyMC for naturally calibrated uncertainty - eliminating the need for the chaos matrix workaround.
- **Mobile app:** React Native wrapper around the same API with push notifications on prediction updates.

Known Limitations

- Render free tier sleeps after 15 min - cold start ~50s affects first-user experience.
- FastF1 rate limit (500 calls/hr) can skip some races during data collection.
- 2025 season data unavailable via current FastF1 version - library update may be needed.
- Practice caches only live locally - cloud storage would make this production-ready.

13. Quick Reference - Common Questions

Question	Answer
What language is the backend?	Python 3.11, using FastAPI + uvicorn.
What language is the frontend?	JavaScript (React 18) with Tailwind CSS.
How many features does the model use?	75 base + 4 chaos scalars + 21 practice features when cached. Up to ~100 total.
How many training samples?	2,979 driver-race entries across 2018-2024 seasons.
What is the model accuracy?	AUC-ROC = 0.9534 (Logistic Regression). 95.3% chance of correctly ranking a winner above a non-winner in any random pair.
Why not use RPM?	Too noisy across different fuel loads. Not a reliable signal from a single lap.
What does the PRACTICE DATA badge mean?	FP1/FP2/FP3 telemetry was precomputed for that GP. Richer features (race pace, trap speeds) are being used.

Question	Answer
Why can't practice data run on Render?	Render free tier has 512 MB RAM. Loading 3 sessions of 10Hz telemetry for 20 drivers exceeds memory. Must run locally.
What is ChaosIndex?	Score 0-1 combining safety car probability, qualifying grid spread, and wind speed. High chaos = wider scenario spread.
What is SC Probability?	Historical fraction of races at that circuit with a safety car or VSC, detected from FastF1 track_status codes 4 and 6.
Where is the model saved?	models/winner_model.joblib - committed to git so Render loads it on deploy.
How do I retrain the model?	Double-click train_model.bat. Takes 20-40 min. New model saves automatically.
How do I deploy changes?	Run git_push.bat. Render and Netlify auto-deploy on push to main.
What years can I predict?	2018-2026. Trained on 2018-2024; 2025 and 2026 use same model on current data.